



Cognitive Science Student Journal
Osnabrück University

Submission template

Great that you submit your scientific paper!

Make sure that you adhere to our **style guide** and fill out our **author form**.

The submission of your scientific paper is divided into three parts within one LaTeX project, for which we provide individual .tex files:

1. Compact Intro
2. Information
3. Text body

1 Compact Intro

How to write the Compact Intro to your work:

Each publication on the Cognitive Science Student Journal website will be introduced with a Compact Intro of the publication. The aim of this Compact Intro is to give prospective students, students and alumni of the Cognitive Science a thematic overview of the publications' content and spark their interest to read the full publication.

We ask you to consider the following guidelines and the example Compact Intro to write your own Compact Intro. The Compact Intro consists of the following three subparts: Summarizing statement, spoiler alert, good to know. After reading through the guidelines and the example, insert your Compact Intro as indicated below.

1. Summarizing statement

- Start with a hook
- Central question (aim of the paper)
- State what the paper is about
- In the end shortly state the roadmap that the reader can expect

2. Spoiler Alert

Summarize the papers conclusion.

3. Good to know

State what prerequisite knowledge is needed to understand the paper. Consider including helpful links.

Tone and Language

- Use simple language and avoid specific scientific terms. If you do use it, provide a short explanation.
- If appropriate, address the reader directly using "you".
- Do not use abbreviations but the full term followed by the abbreviation in brackets.

Example Compact Intro:

A Compact Intro might reads as following.

Summarizing statement

Is it possible to control technology with your imagination? The paper at hand discusses four types of neural networks and their applicability to the domain of visual imagery, i.e. whether you can predict what action a user wants a brain-computer interface (BCI) to perform based on a subject's neuronal data, usually electroencephalographic data (EEG). The paper provides a closer look into

visual imagery tasks and the use of neural networks. The networks compared are standard artificial neural networks (ANNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), as well as spiking neural networks (SNNs). You will be introduced to the general pros and cons of each network and the specific advantages and disadvantages their architecture poses to the field of visual imagery. In the end you will be given a summary and a general recommendation.

Spoiler Alert

For the specific task of translating mental imagery into a command, CNNs are ill-suited; ANNs, RNNs, and SNNs do well but SNNs will potentially outperform the others in the long run.

Good to know

Ideally, you are familiar with the basics of machine learning, what an artificial neural networks is, what different networks there are. You know about electroencephalography (EEG) measurements, what is being measured and how.

Your Compact Intro:

Summarizing statement

Reinforcement learning is an area of machine learning that's been picking up steam since the start of the deep learning revolution. In this paper, we focus on the feature extraction component of reinforcement learning problems, using Atari games as a test bed. We provide an introduction to the basic structure of modern reinforcement learning algorithms, and then we investigate the impact that different vision models have on game performance, which may be useful for practitioners of reinforcement learning.

Spoiler Alert

Training an autoencoder or a CNN from scratch yields reasonable results because of its domain specificity. In contrast, pre-trained models trained on realistic imagery such as the ImageNet dataset are poorly suited to extract helpful visual information from Atari games. They perhaps would do better in more realistic environments.

Good to know

Simply having high-level knowledge of machine learning will be enough to understand this article.

2 Information

Please answer the following questions.

To which Cognitive Science discipline is the paper affiliated to?

Artificial Intelligence

In which seminar did you write that paper? (seminar name, year, name of teacher)

Study Project: Emergent Behaviour in Multi Agent Systems; WS23/24; Teachers: Julius Mayer, Elia Bruni

What kind of paper is it? (e.g. original research, summary, meta paper, experimental sketch, explanation of a paradigm, discussion, ...)

original research

If you wrote your paper within a University course, specify what your task was?

Testing various preprocessing techniques for RL

What is the research question of your paper?

Which one of the three ways to handle visual input, those being end-to-end training, variational autoencoders and pretrained classifiers, work best for RL in atari environments?

3 Text body

Title of the paper: [From Pixels to Actions: Comparing Image Feature Extraction Techniques for Reinforcement Learning](#)

Abstract

This study explores the impact of various image feature extraction techniques on the performance of reinforcement learning (RL) agents. We compared three different types of feature extractors: a custom-built two-layer Convolutional Neural Network (CNN), a selection of pre-trained CNN models, and a Variational Autoencoder (VAE). Our experiments demonstrated that the training the RL-network and the CNN feature extractor end-to-end consistently outperformed the other models in multiple settings, suggesting that simpler models might be more effective due to their optimal balance of complexity and functionality. Using the weights of the pretrained models ResNet and EfficientNet, their efficiency largely depended on the specific setup. Still, they were often more effective than the VAE, indicating the potential advantages of using pre-trained models in certain RL tasks. These findings underline the importance of choosing the right feature extraction strategies to boost the learning efficiency and adaptability of RL agents. The broader implications of these results for developing embodied agents in dynamic and interactive environments are also highlighted, setting a foundation for further investigations in this field.

4 Introduction

In recent years, reinforcement learning (RL) has gained considerable attention for its ability to solve complex tasks through the interaction with dynamic environments (Baker et al., 2019; Vinyals et al., 2019). However, a significant challenge arises when RL models must process high-dimensional visual data, such as images, in real time. Unlike simpler tasks with low-dimensional inputs, image-based environments contain vast amounts of raw pixel information that may be irrelevant or redundant. This high dimensionality can overwhelm traditional RL algorithms, making it difficult for RL agents to identify the most critical features for decision-making (Lange et al., 2012), leading to inefficient learning and poor generalization, particularly when the visual complexity of the environment increases (Rupprecht et al., 2019). To overcome these limitations, various advanced methods have been proposed, including training a Convolutional Neural Network (CNN) feature extractor end-to-end, which directly optimizes the feature extraction layers alongside the policy, as well as leveraging large pre-trained classifier models (Le et al., 2022). Specialized architectures such as variational autoencoders (VAEs) have also been introduced to compress high-dimensional visual inputs into more manageable representations, aiming to improve both learning efficiency and generalization across diverse environments (Janisch et al., 2019; Lange & Riedmiller, 2010).

This paper investigates the use of different visual feature extraction methods in the context of RL, specifically applying these techniques to Atari game environments. We focus on Proximal Policy Optimization (PPO), a widely used RL algorithm known for its balance of stability and efficiency, to train agents across a variety of game scenarios (Schulman et al., 2017; Yu et al., 2022). By comparing end-to-end training, the use of pre-trained CNN models, and VAEs, we aim to identify the trade-offs and performance benefits of each approach in visually complex tasks.

The primary contribution of this work is a detailed empirical evaluation of these feature extraction strategies across multiple Atari games, which differ in visual complexity and dynamics. The results provide insights into the effectiveness and trade-offs between various methods and their impact on both training efficiency and final performance, offering valuable guidance for future applications of RL in visually rich environments.

5 Background

In this section, we start by discussing key RL algorithms and continue with visual feature extraction methods used throughout this paper. We first introduce PPO, the primary RL algorithm utilized, highlighting its advantages in terms of stability and efficiency. We then go into detail about visual feature extraction approaches, including end-to-end training, pre-trained CNN models, and VAEs.

5.1 Proximal Policy Optimization

In our study, we use the Proximal Policy Optimization (PPO) algorithm, which is part of the on-policy family of RL algorithms (Schulman et al., 2017; R. J. Williams, 1992). PPO is a widely used algorithm in the field of RL, particularly noted for its stability and effectiveness in training policies for complex tasks (Yu et al., 2022). Developed to improve upon earlier policy gradient methods, PPO aims to take the largest possible improvement step on a policy while avoiding excessive updates that could lead to performance degradation. This is achieved through the use of a clipped objective function, which prevents the new policy from deviating beyond a certain threshold from the old one, thus maintaining a controlled exploration of the policy space. PPO operates by calculating the ratio of the probability under the new policy to the probability under the old policy for a given action, and then clipping this ratio to lie within a specified range. This clipping mechanism effectively bounds the policy update, promoting a more stable and reliable convergence. Another advantage is its simplicity, which makes it easier to understand and implement (Huang et al., 2022).

5.2 Visual Feature Extraction

In this study, we explore three methods for visual feature extraction in RL tasks involving image-based data. First, we examine the end-to-end training approach, where the feature extraction network is trained with the RL model to optimize task-specific representations. Second, we investigate using pre-trained weights from large-scale image classification models to leverage transfer learning and reduce the computational burden of training from scratch. Finally, we explore Variational Autoencoders (VAEs) to learn compact, low-dimensional feature representations that capture the image data's underlying structure, aiding the RL agent's performance.

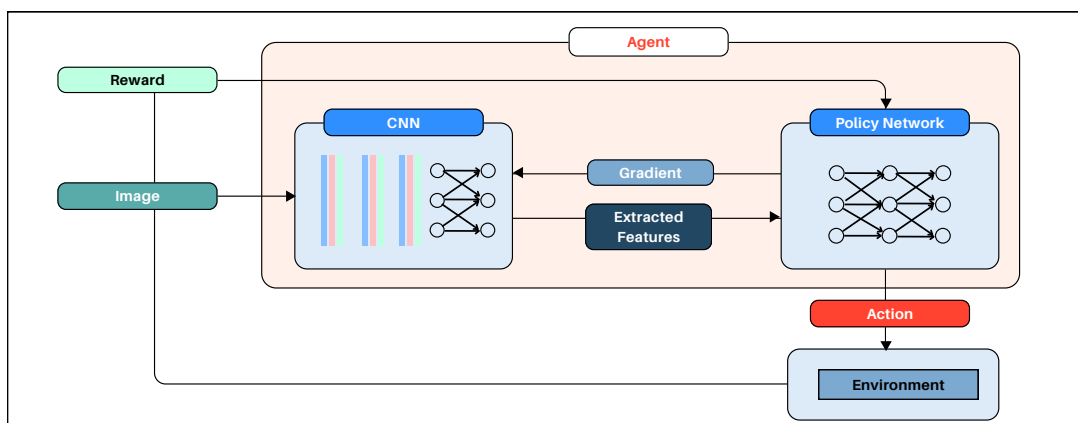


Figure 1: The CNN feature extractor and the policy are trained and updated together

5.2.1 End-to-end Training

The first approach to extracting features from image data is training the CNN at the same time as the policy network, which is also displayed in Figure 1. During this process, the CNN receives raw visual input from the environment and passes it through multiple layers to extract meaningful features. These extracted features are then fed into the policy network, which uses them to decide on the next action. As the entire network is trained together, both the CNN and the policy network are optimized to improve performance in the RL task (Le et al., 2022).

End-to-End training is the most intuitive approach, as we do not have to use or integrate different architectures. We simply train both the policy network and the layers for feature extraction as one big network (Le et al., 2022). CNNs are particularly suited to this problem because they take advantage of correlations between locally grouped values in the images, and features extracted in this manner are invariant to location. Furthermore, they can automatically learn hierarchical feature representations—from simple edges and textures to complex shapes and objects—which enhances the model's ability to interpret and act upon visual data effectively (LeCun et al., 2015). For example, a CNN recognizing a ball in Pong will use the same basic features to identify it regardless of where on the screen it is found.

5.2.2 Pre-trained Weights

Another common strategy to extract features for an image processing component involves using a pre-trained classifier model (Pan & Yang, 2010; Schwarzer et al., 2021), as can be seen in Figure 2.

This model has already been trained on a vast dataset to extract features from raw images. Instead of training a CNN from scratch, a frozen version of this model is re-used to extract features from the agents visual perceptions. To do so, the final layer of the pre-trained model is removed. The remaining layers are repurposed for the new task. The image is passed through the pre-trained network and the resulting feature vectors are used as state input for the RL algorithm (E. Williams & Polydoros, 2024). In the case of a classification model, the final layer would be a fully connected layer outputting the probability distribution for the different classes of images the model was originally trained on. This carries the benefit of precluding training of the vision model while leveraging the predictive power of a model that may have been very expensive to train.

In our experiments, we used the ResNet50 and EfficientNet architectures (He et al., 2015; Tan & Le, 2019), which were trained on the ImageNet dataset, containing 1.2 million training images. The first of these models has 25.5 million parameters, while the 2nd has 7.6 million parameters.

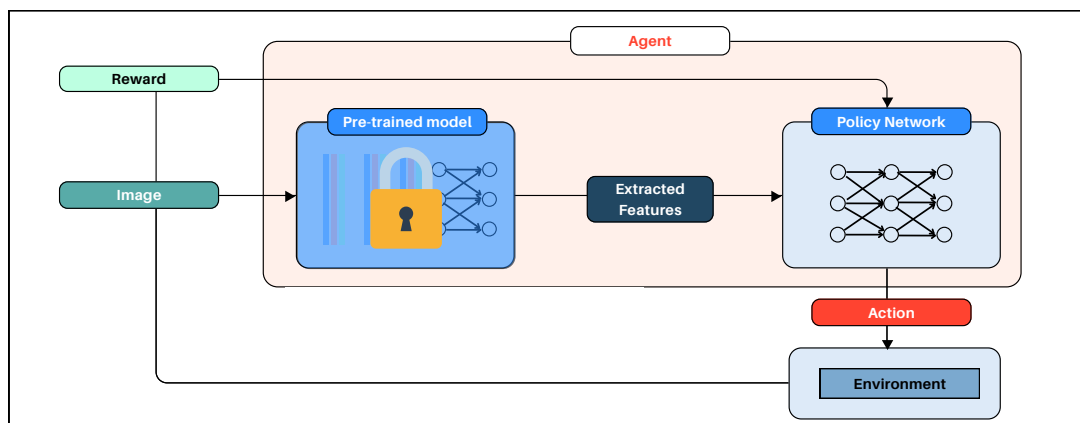


Figure 2: The agent uses pretrained weights from large classifier networks as a feature extractor

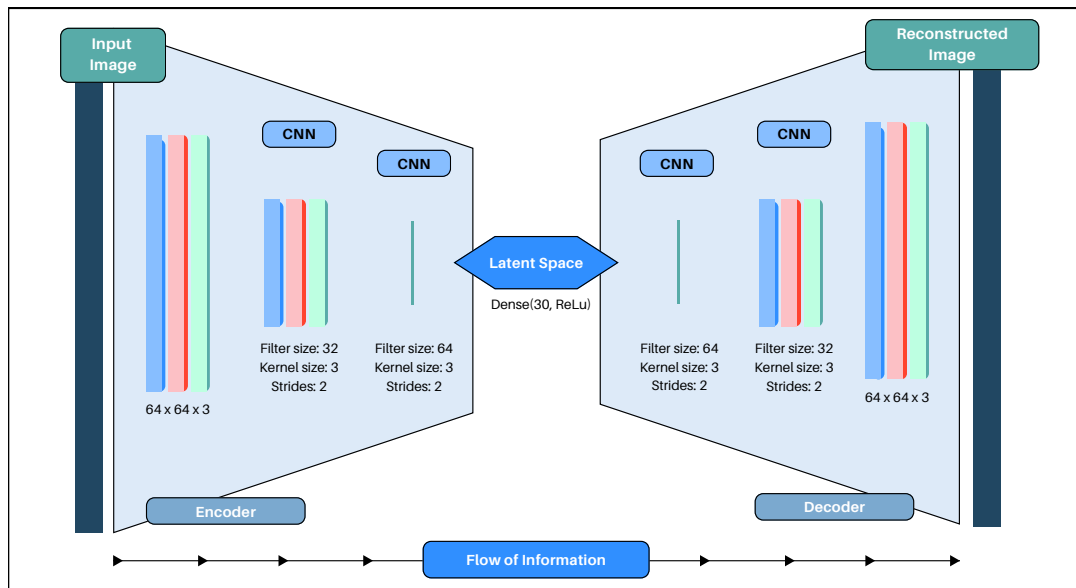


Figure 3: Autoencoder Network Architecture

5.2.3 Variational Autoencoders

The third approach of extracting features involves using a variational autoencoder (VAE) (Kingma & Welling, 2013). Autoencoders, as displayed in Figure 3, are applied in unsupervised learning and consist of an encoder and a decoder with an intermediate latent space, commonly referred to as bottleneck (Schmidhuber, 2015). In convolutional autoencoders, the encoder compresses the input into a lower-dimensional representation by progressively reducing the kernel size across convolutional blocks. The most compressed representation is stored in the latent space and holds the smallest possible summary of the input data. The decoder then mirrors the encoder's structure to reconstruct the original input from the latent space. By upsampling through convolutional blocks with increasing kernel sizes a reconstruction loss is minimized (Schmidhuber, 2015; Zhang, 2018). However, standard autoencoders do not account for uncertainty in the latent space, which can lead to overfitting and poor generalization. Instead of learning a generalizable representation, the model might memorize the training data, achieving high accuracy on familiar images from the training set but struggling with unseen data. Furthermore, it might use tiny changes in its latent space representation to represent big changes in the image, which makes it increasingly difficult for the policy network to understand the latent space representations.

To address this, VAEs add stochasticity by encoding images as a distribution over the latent space to be sampled from before decoding, ensuring that the model learns to generate more diverse and coherent outputs. The regularization term, often a Kullback-Leibler (KL) divergence, is added to the loss function to enforce the latent space to resemble a standard Gaussian distribution, which promotes a smooth and continuous representation that allows for better interpolation and sampling of new data points (Kingma & Welling, 2013, 2019). Because we are ultimately interested in feature extraction, we use only the encoder component in conjunction with PPO at inference time, similar to using the pretrained classifier weights. This brings a significant theoretical performance benefit compared to end-to-end training, as we do not need to perform updates on the weights of the CNN, as it is pre-trained and remains static during the training of the PPO agent.

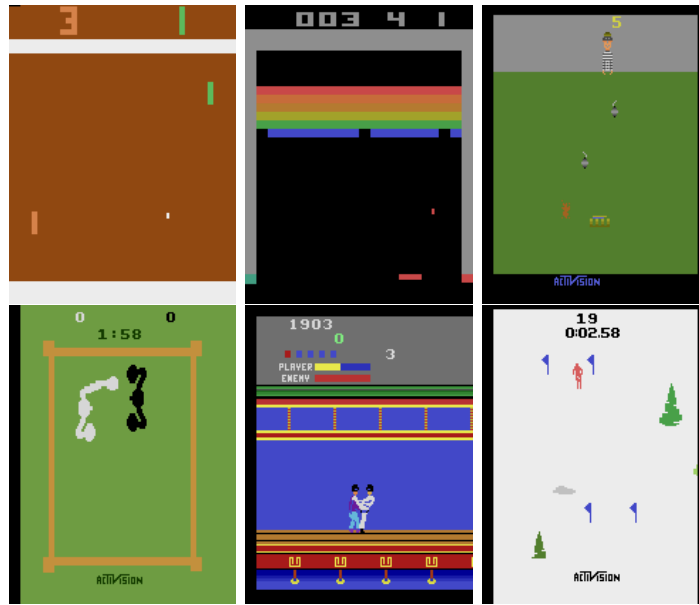


Figure 4: Example observations from Gymnasium Atari environments: Pong, Breakout, Kaboom, Boxing, Kung Fu Master, Skiing

6 Methodology

In our experiments, we used Farama's Gymnasium¹ library to provide the environments for the RL agent to interact with, namely six Atari games: Breakout, Boxing, Skiing, Kung Fu Master, Kaboom, and Pong, which can be seen in Figure 4 (Towers et al., 2024). These six environments represented a limited range of images possible under the Atari console technology, from highly abstract (Pong) to imitating features of naturalistic scenes (Skiing). The PPO agent is trained using Stable Baselines3's implementation of the algorithm, whose modularity allows the easy use of custom feature extractors (Raffin et al., 2021).

We use Stable Baseline3's standard Atari preprocessing before providing images to the feature extractors. This includes downscaling images to 84 pixel squares and converting the 3-channel images to grayscale ones. Additionally, we incorporate frame stacking, which vertically concatenates four sequential frames from the Atari games. This technique provides information about the motion of objects that must be interpreted by the feature extractors (such as the direction and speed of a ball). All PPO models are trained over 3,000,000 timesteps, using a learning rate of $2.5e-4$, a batch size of 128, and a PPO ϵ clipping value of 0.1. The entropy coefficient, which helps to regularize, is set to 0.01. Furthermore, there are two fully connected layers of width 128 between the feature extractor and the value and policy networks.

In this study, we test three different approaches: end-to-end training, using the weights of the encoder from a VAEs and leveraging pretrained weights from ResNet-50 and EfficientNet architectures. For the end-to-end approach, the CNNs are trained with varying numbers of hidden units in the final/bottleneck layer: 256, 512, and 1024. In the results section, we append the number of hidden units to the model name for clarity. For the VAE-based approach, we pretrain the VAE with a latent space size of 300, using three Conv2D layers in the encoder and three ConvTranspose2D layers in the decoder. The pretraining dataset is generated by sampling images from the environment, where 10,000 random actions are performed, and their outputs recorded. Lastly, for the pretrained

¹<https://gymnasium.farama.org/index.html>

approach, we fine-tune models initialized with weights from ResNet-50 and EfficientNet, allowing us to leverage the generalization capabilities of these pre-trained architectures.

All experiments were conducted on a single machine equipped with an Intel Core™ i9-12900K processor, 128 GB of system RAM, and a GeForce RTX 3090 GPU with 24 GB of video RAM.

7 Results

Our results in Figure 5 show the successful training of the VAEs for each environment, displaying the loss over each training epoch. The further drop in loss after epoch 19 can be explained with the drop of the learning rate in this epoch, as this allowed the model to learn a more precise reconstruction policy. In all cases, the encoders of the VAEs manage to learn a meaningful latent space representation, from which the decoders are able to largely reconstruct the image.

Figure 6 shows a comparative analysis of various models' performances across several Atari game environments, using smoothed value metrics plotted against training steps (scaled by $1e6$). Distinct trends in model performance are evident, with games like "BreakoutNoFrameskip-v4" showing a steady improvement in scores as training progresses, indicative of effective learning patterns in simpler game dynamics. Conversely, environments such as "SkiingNoFrameskip-v4" display more erratic performance, suggesting a challenge in mastering more complex or stochastic game mechanics.

In the cases of "Kaboom-v5" and "BreakoutNoFrameskip-v5", end-to-end training clearly beats the other approaches with a much higher average reward per epoch over time. Also in general, the CNN models, especially with higher parameters (e.g., CNN 1024), perform better, showing peaks in "KungFuMaster-NoFrameskip-v4" and sustaining high scores in "BoxingNoFrameskip-v4". Interestingly, in "BoxingNoFrameskip-v4" the figure shows that the end-to-end training with the smallest representation size reaches the highest average reward per epoch, indicating that it is more important for the agent to understand the underlying game mechanics, than the choice of preprocessing. This corresponds with the results for "Skiing-NoFrameskip-v4", where the choice of architecture seems to have less of an impact on overall performance, as end-to-end training with a layer size of 256, as well as using the VAE- and EfficientNet-architectures seemed to work, while larger CNN layers and ResNet-50 failed to result in a meaningful performance. This highlights the game's complexity and the possible need for tailored strategies or enhanced feature extraction techniques.

With regard to the VAE, the results were a bit more mixed. While in some instances, the results are

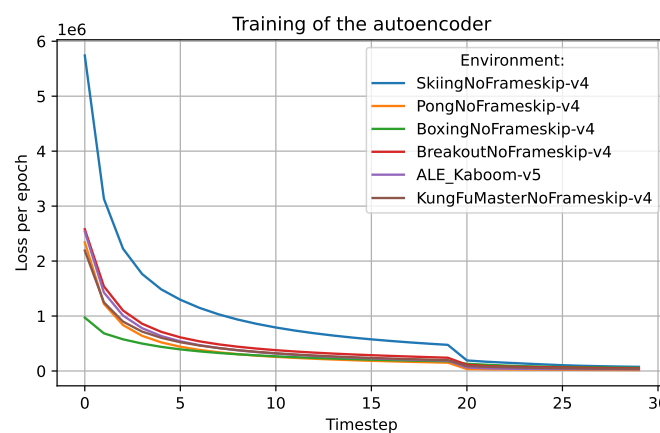


Figure 5: Training results in different Gymnasium Atari environments using end-to-end training (CNN 256, 5123, 1024), pretrained weights (EfficientNet, ResNet-50) and a VAE

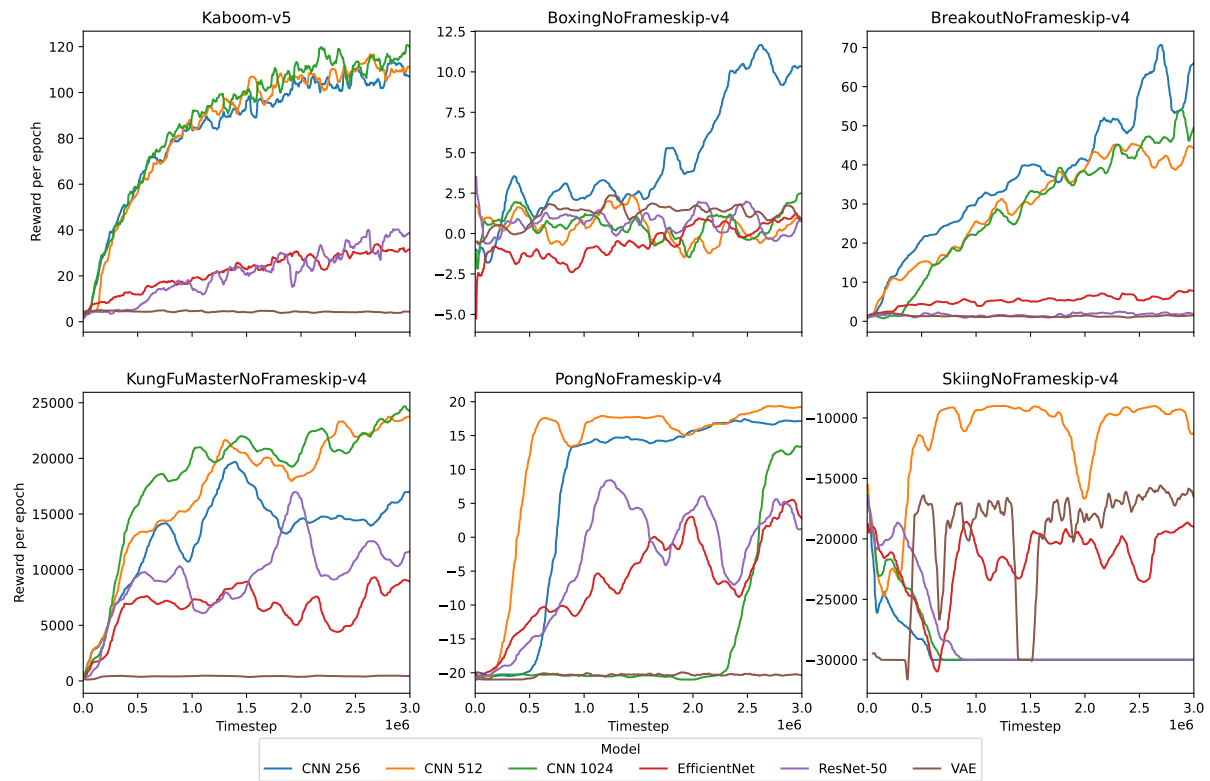


Figure 6: Training results in different Gymnasium Atari environments using end-to-end training (CNN 256, 5123, 1024), pretrained weights (EfficientNet, ResNet-50) and a VAE

similar to some of the end-to-end training runs, in others, performance is significantly worse. Overall, our results show that end-to-end training is consistently amongst the best performing training runs. At the same time, during training, the sampling speed of end-to-end training is nearly 50 percent slower compared to using the fixed weights of the pre-trained encoder of the VAE. This can be explained by the fact that during end-to-end training the backward pass also has to be performed for the CNN weights, which is computationally expensive. We will try to explain these results in the following discussion chapter.

8 Discussion

Overall, we succeeded in comparing the different approaches and were able to show that using end-to-end training often is the best viable options when it comes to abstract environments such as the ones shown above. While the pre-trained model often performed worse than the other approaches, this was still meaningful, as it demonstrated clearly that the selected approach is highly dependent on the specific environment. In this case, we assume that the pre-trained ResNet and EfficientNet weights did not always return meaningful representations, as they were trained on the ImageNet dataset, containing mostly pictures from the real world. This means, that the images from the Atari environments could mostly be out of distribution, leading to the PPO model not being able to learn a consistent policy. This does not entirely rule out the viability of this approach, as it might still be valid when dealing with RL tasks that use real-life visual data.

When choosing between VAE and the other tested options, the decision largely depends on the specific environment and task. If a comprehensive training dataset that captures most of the expected

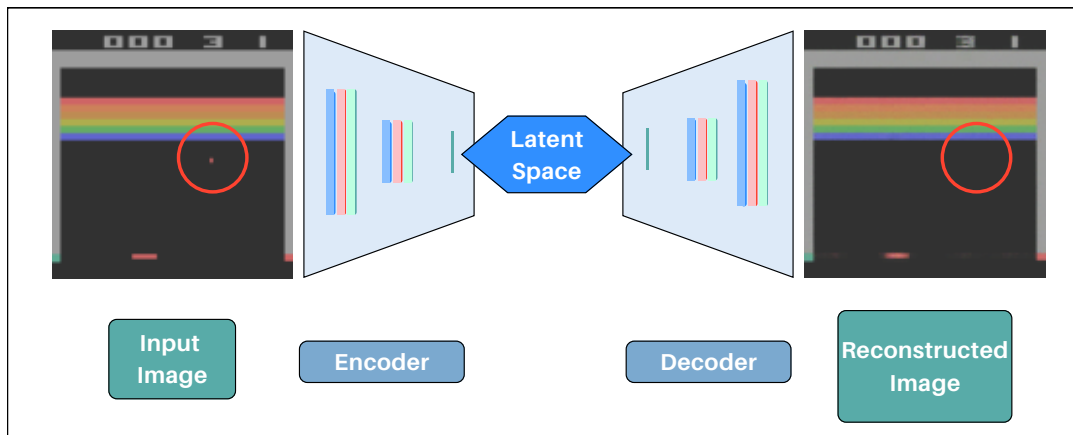


Figure 7: The VAE struggles with details like the red ball in the original image (left), as it does not show up in the reconstructed image (right)

visual states is available, using a VAE and freezing the encoder weights during the RL training phase can save time. The latter also becomes increasingly significant when training times are long, as this approach offers a performance advantage by reducing computational load, as it eliminates the need to train the CNN weights during RL training. However, the importance of visual details plays a crucial role. Autoencoders sometimes fail to capture fine details, such as in games like Pong and Breakout, where small but critical elements (like a one-pixel ball) may not be accurately represented in the embedding space. Figure 7 illustrates this issue, showing how the VAE fails to reconstruct the ball, as its small size contributes minimally to the reconstruction loss. In scenarios where visual states are unpredictable or when these details are vital, end-to-end training may be the better option.

9 Conclusion

In this paper, we explored and compared different image feature extraction methods for RL tasks within visually complex environments. Our results indicate that simpler, custom-built CNN architectures tend to outperform more complex pre-trained models and VAEs in multiple Atari game settings. Specifically, end-to-end CNN training demonstrated significant benefits, particularly in environments where fine visual details are critical, such as games involving small but important objects like the ball in Pong. On the other hand, while pre-trained models like ResNet and EfficientNet showed weaker performance, their use in real-world visual tasks remains a promising area for further exploration.

The trade-offs between using VAEs and end-to-end training were also highlighted. VAEs can be beneficial in environments with well-defined and predictable visual states, offering computational efficiency by freezing the vision encoder weights during training. However, in dynamic and visually detailed environments, end-to-end training may provide superior performance by directly adapting the feature extraction process to the task at hand.

In summary, the effectiveness of each feature extraction technique is highly context-dependent, influenced by both the complexity of the environment and the nature of the task. These findings offer practical insights into selecting appropriate methods for RL in image-based domains and suggest directions for future work, such as refining and using new autoencoders architectures for more detailed visual representations or exploring task-specific pre-trained models for RL applications.

Data Availability

The datasets and code used in this study are available in [this GitHub Repository](#)². This includes the full implementation of the RL models, feature extraction methods, and all training and evaluation results. Further details and instructions for reproducing the experiments can also be found in the repository.

Acknowledgements

This research was conducted within the scope of MicrocosmAI, which made this work possible. For further information and access to the code, please visit microcosm.ai.

References

- Baker, B., Kanitscheider, I., Markov, T., Wu, Y., Powell, G., McGrew, B., & Mordatch, I. (2019). Emergent tool use from multi-agent autocurricula. *arXiv preprint arXiv:1909.07528*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition.
- Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., & AraÅšjo, J. G. (2022). Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274), 1–18.
- Janisch, J., Pevný, T., & Lisý, V. (2019). Classification with costly features using deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 3959–3966.
- Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- Kingma, D. P., & Welling, M. (2019). An Introduction to Variational Autoencoders. *Foundations and Trends® in Machine Learning*, 12(4), 307–392. <https://doi.org/10.1561/22000000056>
- Lange, S., & Riedmiller, M. (2010). Deep auto-encoder neural networks in reinforcement learning. *The 2010 international joint conference on neural networks (IJCNN)*, 1–8.
- Lange, S., Riedmiller, M., & Voigtländer, A. (2012). Autonomous reinforcement learning on raw visual input data in a real world application. *The 2012 international joint conference on neural networks (IJCNN)*, 1–8.
- Le, N., Rathour, V. S., Yamazaki, K., Luu, K., & Savvides, M. (2022). Deep reinforcement learning in computer vision: A comprehensive survey. *Artificial Intelligence Review*, 1–87.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436–444.
- Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359. <https://doi.org/10.1109/TKDE.2009.191>
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- Rupprecht, C., Ibrahim, C., & Pal, C. J. (2019). Finding and visualizing weaknesses of deep reinforcement learning agents. *arXiv preprint arXiv:1904.01318*.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural networks*, 61, 85–117.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms.
- Schwarzer, M., Rajkumar, N., Noukhovitch, M., Anand, A., Charlin, L., Hjelm, R. D., Bachman, P., & Courville, A. C. (2021). Pretraining representations for data-efficient reinforcement learning. *Advances in Neural Information Processing Systems*, 34, 12686–12699.
- Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. *International conference on machine learning*, 6105–6114.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*.

²<https://github.com/microcosmAI/RL-Vision-Comparison>

- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782), 350–354.
- Williams, E., & Polydoros, A. (2024). Pretrained visual representations in reinforcement learning. <https://arxiv.org/abs/2407.17238>
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8, 229–256.
- Yu, C., Velu, A., Vinitisky, E., Gao, J., Wang, Y., Bayen, A., & Wu, Y. (2022). The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35, 24611–24624.
- Zhang, Y. (2018). A better autoencoder for image: Convolutional autoencoder. *ICONIP17-DCEC*. Available online: http://users.cecs.anu.edu.au/Tom.Gedeon/conf/ABCs2018/paper/ABCs2018_paper_58.pdf (accessed on 23 March 2017).